



Namal University, Mianwali
DEPARTMENT OF COMPUTER SCIENCES

Virtual Fitness Trainer

Database Design Document

Version 2.0

CSC-271 — Database Systems

Submitted By:

Name	Registration No.
Muhammad Wasif	NUM-BSCS-2024-59
Hifza Bibi	NUM-BSCS-2024-27

Submission Date:

14st June, 2026

Revision History

Date	Version	Description of Changes	Approved By
15 April, 2026	1.0	Initial version — Complete Database Design Document (Milestone 2: ERD Design)	Instructor
21 June, 2026	2.0	Relational schema conversion, normalization, DDL/DML implementation, stored procedures, GUI development and integration	Instructor

Contents

Revision History	1
1 Project Overview	4
1.1 Introduction	4
1.2 Problem Statement	4
1.3 Project Objectives	4
1.4 GitHub Repository Link	5
2 Detailed Database Design	6
2.1 Entities	6
2.2 Data Dictionary	7
2.2.1 User	7
2.2.2 Health_Condition	10
2.2.3 Injuries	10
2.2.4 Food_Allergy	10
2.2.5 Login_Activity	11
2.2.6 Exercise	12
2.2.7 Exercise_Video	13
2.2.8 Workout_Plan	14
2.2.9 Workout_Day	15
2.2.10 Workout_Plan_Exercise	16
2.2.11 Diet_Plan	17
2.2.12 Recommendation	18
2.3 Business Rules	18
2.4 Entity Relationship Diagram	19
3 ERD to Relational Schema Conversion	21
3.1 Conversion Methodology	21
3.2 Resulting Relational Schema	21
4 Functional Dependencies and Normalization	23

4.1	Functional Dependencies	23
4.2	Normalization	27
4.2.1	User (ERD Entity: User)	27
4.2.2	Profile_Audit_Log (ERD Entity: Profile Audit Log)	27
4.2.3	Login_Activity	28
4.2.4	Exercise (ERD Entity: Exercise)	28
4.2.5	Workout_Plan, Workout_Day, Workout_Plan_Exercise	28
4.2.6	Diet_Plan, Diet_Day, Meal, Recipe	28
4.2.7	Meal_Ingredient	28
4.2.8	Progress_Log and Progress_Log_Exercise	29
4.2.9	Recommendation	29
4.2.10	Normalization Summary	29
5	Database Implementation	31
5.0.1	Triggers	31
5.0.2	Scalar Functions	32
5.0.3	Stored Procedures	32
6	GUI Design and Development	34
6.1	Technology Stack	34
6.2	Application Architecture	34
6.3	GUI Features and Screens	35
6.3.1	User Registration and Login	35
6.3.2	Dashboard	36
6.3.3	AI Workout Plan Generation	37
6.3.4	AI Diet Plan Generation	37
6.3.5	Exercise Library	38
6.3.6	Weekly Progress Logging	39
6.3.7	Progress Reports and PDF Export	39
6.3.8	Recommendations	40
6.4	Database Connectivity	40
6.5	Security Implementation	41
7	References	42
A	Appendix: AI Usage and Prompts	43

Chapter 1

Project Overview

1.1 Introduction

Virtual Fitness Trainer (VFT) is an intelligent web-based fitness guidance system that provides personalized workout and diet plans using Artificial Intelligence. The system is designed to help individuals achieve their fitness goals by considering their personal health data, fitness level, preferences, and any existing medical conditions.

This application targets fitness enthusiasts, beginners, and health-conscious individuals who want customized guidance without the need for expensive personal trainers or generic one-size-fits-all programs. The system supports comprehensive features including AI-powered plan generation via Google Gemini AI, exercise video demonstrations with embedded YouTube players, weekly progress tracking, adaptive recommendations, and exportable PDF progress reports.

1.2 Problem Statement

In today's fast-paced world, people struggle to maintain consistent fitness routines due to lack of personalized guidance, time constraints, and limited access to professional trainers. Existing fitness applications often provide generic workout and diet plans that do not consider individual health conditions, body metrics, or specific goals, which can lead to ineffective results or even injuries.

Moreover, manual tracking of progress, lack of proper form guidance through videos, and absence of adaptive plan modification based on user performance are major challenges. There is a clear need for an intelligent, accessible, and personalized fitness solution that can safely guide users according to their unique requirements while continuously adapting to their progress.

1.3 Project Objectives

General Objective: To develop a comprehensive database-backed Virtual Fitness Trainer system that delivers personalized, AI-powered workout and diet plans with effective progress tracking and adaptive recommendations.

Specific Objectives:

- Enable users to create detailed profiles including health conditions, body measurements, and fitness goals for accurate personalization.
- Generate customized workout and diet plans using Google Gemini AI based on user data and preferences, with automatic fallback to template plans.
- Provide embedded video demonstrations for exercises to ensure correct form and reduce injury risk.
- Allow users to log weekly progress and generate insightful weekly and monthly reports exportable as PDF files.
- Implement an adaptive recommendation system that modifies plans based on user performance and feedback.
- Maintain complete audit logs and progress history for better user accountability and system improvement.
- Ensure data integrity and efficient retrieval through stored procedures and triggers.
- Build a secure, fully integrated GUI using Python Flask with all database access restricted to stored procedures.

1.4 GitHub Repository Link

<https://github.com/wasif-cmd/Virtual-Fitness-Trainer>

Chapter 2

Detailed Database Design

2.1 Entities

Sr.	Entity Name	Description
1	User	Central entity representing registered users of the system, storing personal and fitness-related information.
2	Health_Condition	Stores individual health conditions per user (decomposed from User multivalued attribute during normalization).
3	Injuries	Stores individual injuries or physical limitations per user.
4	Food_Allergy	Stores individual food allergies per user (decomposed from User multivalued attribute).
5	Login_Activity	Records user login sessions, device information, and security-related login data.
6	Profile_Audit_Log	Maintains header records of changes made to user profiles for security and auditing.
7	Audit_Field_Change	Stores field-level details of each profile change (decomposed from Profile_Audit_Log).
8	Exercise	Stores information about individual exercises including name, difficulty, tips, and category.
9	Muscle_Group	Stores muscle groups targeted by each exercise (decomposed from Exercise multivalued attribute).
10	Target_Area	Stores specific target areas for each exercise (decomposed from Exercise multivalued attribute).

Sr.	Entity Name	Description
11	Exercise_Video	Contains video URLs and embedded codes for exercise demonstration.
12	Workout_Plan	Represents personalized workout plans generated for users based on their goals.
13	Workout_Day	Defines individual days within a workout plan including focus area and session details.
14	Workout_Plan_Exer	Associative entity linking exercises to specific days in a workout plan.
15	Diet_Plan	Represents personalized diet plans generated according to user's caloric needs.
16	Diet_Day	Defines daily meal structure within a diet plan.
17	Meal	Individual meals within a diet day containing nutritional information.
18	Recipe	Detailed cooking instructions and preparation information for meals.
19	Ingredient	Stores nutritional information of food ingredients used in recipes.
20	Meal_Ingredient	Associative entity linking ingredients to recipes with quantity.
21	Progress_Log	Records user's weekly progress including weight, measurements, and workout completion.
22	Progress_Log_Exerc	Tracks performance details for individual exercises within a progress log entry.
23	Recommendation	AI-generated recommendations for plan adjustments or alternatives.

2.2 Data Dictionary

2.2.1 User

Sr.	Attribute	Data Type	Constraint	Description
1	UserID	INT UNSIGNED	PK, NOT NULL, AUTO_INCREMENT	Unique identifier.
2	Name	VARCHAR(50)	NOT NULL	Full name of the user.
3	Email	VARCHAR(255)	NOT NULL, UNIQUE	Email for login and notifications.
4	Password_Hash	VARCHAR(255)	NOT NULL	Bcrypt-hashed password.
5	Birth_Date	DATE	NOT NULL	Date of birth.
6	Gender	ENUM	NOT NULL	Male / Female / Other.
7	Height	DECIMAL(5,2)	NOT NULL	Height in centimetres (120–250).
8	Weight	DECIMAL(6,2)	NOT NULL	Weight in kilograms (30–300).
9	Body_Type	ENUM	NOT NULL	Slim / Average / Athletic / Overweight.
10	Activity_Level	ENUM	NOT NULL	Sedentary to Extra Active.
11	Primary_Goal	ENUM	NOT NULL	Fitness goal (e.g., Weight Loss).
12	Target_Weight	DECIMAL(6,2)	NULL	Desired target weight (kg).
13	Timeline	TINYINT	NULL	Months to achieve goal (1–12).
14	Current_Fitness_Level	ENUM	NOT NULL	Beginner / Intermediate / Advanced.
15	Workout_Frequency	TINYINT	NOT NULL	Workout days per week (1–7).

Sr.	Attribute	Data Type	Constraint	Description
16	Dietery_Preference	ENUM	NOT NULL	Vegetarian / Vegan / Non-Veg / Pescatar-ian.
17	Age	TINYINT UNSIGNED	NULL	Derived from Birth_Date.
18	BMI	DECIMAL(5,2)	NULL	Body Mass Index (calculated).
19	BMR	DECIMAL(8,2)	NULL	Basal Metabolic Rate (calculated).
20	TDEE	DECIMAL(8,2)	NULL	Total Daily Energy Expenditure (calculated).
21	Incorrect_Login_A	TINYINT UNSIGNED	NOT NULL DEFAULT 0	Failed login counter.
22	Is_Locked	TINYINT(1)	NOT NULL DEFAULT 0	Account lock flag.
23	Locked_Until	DATETIME	NULL	Lock expiry time.
24	Is_Verified	TINYINT(1)	NOT NULL DEFAULT 0	Email verification flag.
25	Created_At	DATETIME	NOT NULL, DEFAULT NOW()	Account creation timestamp.
26	Updated_At	DATETIME	NOT NULL, ON UPDATE NOW()	Last profile update timestamp.

2.2.2 Health_Condition

Sr.	Attribute	Data Type	Constraint	Description
1	HealthConID	INT UNSIGNED	PK, AUTO_INCREMENT	Unique identifier.
2	UserID	INT UNSIGNED	FK → User	References the user.
3	HealthCondition	ENUM	NOT NULL, UNIQUE(UserID)	Specific health condition.

2.2.3 Injuries

Sr.	Attribute	Data Type	Constraint	Description
1	InjuryID	INT UNSIGNED	PK, AUTO_INCREMENT	Unique identifier.
2	UserID	INT UNSIGNED	FK → User	References the user.
3	Injury	VARCHAR(255)	NOT NULL	Description of the injury.

2.2.4 Food_Allergy

Sr.	Attribute	Data Type	Constraint	Description
1	FoodAllergyID	INT UNSIGNED	PK, AUTO_INCREMENT	Unique identifier.
2	UserID	INT UNSIGNED	FK → User	References the user.
3	Food_Allergy	VARCHAR(100)	NOT NULL, UNIQUE(UserID)	Name of the food allergy.

2.2.5 Login_Activity

Sr.	Attribute	Data Type	Constraint	Description
1	LoginID	INT UNSIGNED	PK, AUTO_INCREMENT	Unique login record ID.
2	UserID	INT UNSIGNED	FK → User, NULL	NULL when email not found.
3	Email	VARCHAR(255)	NOT NULL	Email used during login attempt.
4	Remember_Me	TINYINT(1)	DEFAULT 0	Remember-me option flag.
5	IsFailed	TINYINT(1)	NOT NULL DEFAULT 0	Failed login indicator.
6	Login_Time	DATETIME	NOT NULL DEFAULT NOW()	Login timestamp.
7	IP_Address	VARCHAR(45)	NOT NULL	IPv4 or IPv6 address.
8	Device_Category	ENUM	NOT NULL DEFAULT 'Unknown'	Desktop / Mobile / Tablet / Unknown.
9	OS_Name	VARCHAR(50)	NULL	Operating system name.
10	Browser_Name	VARCHAR(50)	NULL	Browser name.

2.2.6 Exercise

Sr.	Attribute	Data Type	Constraint	Description
1	ExerciseID	INT UNSIGNED	PK, AUTO_INCREMENT	Unique exercise identifier.
2	Exercise_Name	VARCHAR(100)	NOT NULL, UNIQUE	Name of the exercise.
3	Exercise_Category	ENUM	NOT NULL	Cardio / Strength / Flexibility / etc.
4	Difficulty_Level	ENUM	NOT NULL	Beginner / Intermediate / Advanced.
5	Equipment_Needed	ENUM	NOT NULL DEFAULT 'None'	Equipment required.
6	Calories_Burned_1	DECIMAL(6,2)	NULL	Estimated calories per rep.
7	Description	TEXT	NULL	How to perform the exercise.
8	Tips	TEXT	NULL	Helpful execution tips.
9	Common_Mistakes	TEXT	NULL	Common mistakes to avoid.

2.2.7 Exercise_Video

Sr.	Attribute	Data Type	Constraint	Description
1	VideoID	INT UNSIGNED	PK, AUTO_INCREMENT	Unique video record ID.
2	ExerciseID	INT UNSIGNED	FK → Exercise	References the parent exercise.
3	URL	VARCHAR(2048)	NOT NULL	Direct YouTube URL.
4	Embedded_Code	TEXT	NULL	HTML iframe embed code.
5	Language	VARCHAR(10)	NOT NULL	Video language (e.g., en, ur).

2.2.8 Workout_Plan

Sr.	Attribute	Data Type	Constraint	Description
1	WorkoutPlanID	INT UNSIGNED	PK, AUTO_INCREMENT	Unique plan identifier.
2	UserID	INT UNSIGNED	FK → User	References the plan owner.
3	Title	VARCHAR(150)	NOT NULL	Title of the workout plan.
4	Goal	ENUM	NOT NULL	Fitness goal for the plan.
5	Plan_Type	ENUM	NOT NULL DEFAULT 'Home'	Home / Gym.
6	Is_Active	TINYINT(1)	NOT NULL DEFAULT 1	1 = Active, 0 = Inactive.
7	Start_Date	DATE	NOT NULL	Start date of the plan.
8	End_Date	DATE	NULL	Expected end date.
9	Created_At	DATETIME	NOT NULL DEFAULT NOW()	Plan creation timestamp.

2.2.9 Workout_Day

Sr.	Attribute	Data Type	Constraint	Description
1	WorkoutDayID	INT UNSIGNED	PK, AUTO_INCREMENT	Unique day identifier.
2	WorkoutPlanID	INT UNSIGNED	FK → Workout_Plan	References parent plan.
3	Day_Number	TINYINT UNSIGNED	NOT NULL, 1–7	Sequential day number.
4	Day_Name	ENUM	NOT NULL	Monday through Sunday.
5	Focus_Area	VARCHAR(100)	NULL	Primary muscle group for the day.
6	Session_Duration	SMALLINT UNSIGNED	NULL	Estimated session length (min).
7	Calories_Estimate	SMALLINT UNSIGNED	NULL	Estimated calories burned.
8	Is_Rest_Day	TINYINT(1)	NOT NULL DEFAULT 0	Rest/recovery day flag.
9	Notes	TEXT	NULL	Additional instructions.

2.2.10 Workout_Plan_Exercise

Sr.	Attribute	Data Type	Constraint	Description
1	PlanExerciseID	INT UNSIGNED	PK, AUTO_INCREMENT	Unique assignment ID.
2	WorkoutDayID	INT UNSIGNED	FK → Workout_Day	References the workout day.
3	ExerciseID	INT UNSIGNED	FK → Exercise	References the exercise.
4	Setss	TINYINT UNSIGNED	NOT NULL DEFAULT 3	Number of sets.
5	Reps	TINYINT UNSIGNED	NULL	Number of repetitions.
6	Rest_Time_Second	SMALLINT UNSIGNED	NULL	Rest between sets (seconds).
7	Intensity	ENUM	NOT NULL DEFAULT 'Medium'	Low / Medium / High.
8	Order_In_Day	TINYINT UNSIGNED	NOT NULL DEFAULT 1	Exercise order in the day.

2.2.11 Diet_Plan

Sr.	Attribute	Data Type	Constraint	Description
1	DietPlanID	INT UNSIGNED	PK, AUTO_INCREMENT	Unique diet plan ID.
2	UserID	INT UNSIGNED	FK → User	Plan owner.
3	Title	VARCHAR(150)	NOT NULL	Diet plan title.
4	Plan_Type	ENUM	NOT NULL	Weight Loss / Muscle Gain / Maintenance.
5	Goal	ENUM	NOT NULL	Fitness goal.
6	Is_Active	TINYINT(1)	NOT NULL DEFAULT 1	Active status flag.
7	Cultural_Preference	ENUM	NULL	Pakistani / Western / Mediterranean / Other.
8	Start_Date	DATE	NOT NULL	Plan start date.
9	End_Date	DATE	NULL	Plan end date.
10	Created_At	DATETIME	NOT NULL DEFAULT NOW()	Creation timestamp.

2.2.12 Recommendation

Sr.	Attribute	Data Type		Description
1	RecommendationID	INT	UN-SIGNED (PK)	Unique identifier.
2	UserID	INT	UN-SIGNED (FK)	References the user.
3	Recommendation_Type	ENUM		Workout / Diet / General.
4	Trigger_Reason	TEXT	NOT NULL	Why this recommendation was triggered.
5	Recommendation_Date	DATE	NOT NULL	Date generated.
6	Status	ENUM		Pending / Accepted / Rejected.
7	Suggested_Changes	TEXT		Proposed plan modifications.
8	Source_WorkoutPlanID	INT	UN-SIGNED (FK, NULL)	Source workout plan reference.
9	Source_DietPlanID	INT	UN-SIGNED (FK, NULL)	Source diet plan reference.
10	Suggested_WorkoutPlanID	INT	UN-SIGNED (FK, NULL)	Suggested replacement workout plan.
11	Suggested_DietPlanID	INT	UN-SIGNED (FK, NULL)	Suggested replacement diet plan.

2.3 Business Rules

1. Each **User** must have a unique email address and a securely bcrypt-hashed password.
2. A user can have only one **active Workout_Plan** and one **active Diet_Plan** at any

- given time (enforced by triggers `trg_WorkoutPlan_DeactivateOld` and `trg_DietPlan_DeactivateOld`).
3. A **Workout_Plan** must contain exactly seven **Workout_Day** entries (one per day of the week), at least some of which include exercises.
 4. The system shall automatically calculate and store BMI, BMR, and TDEE whenever a user updates their height, weight, or activity level (enforced by trigger `trg_User_Before_Update`).
 5. Workout and diet plans must be generated considering the user's health conditions, injuries, dietary preferences, and food allergies.
 6. A **Progress_Log** entry can only be created once per week per user. Users cannot log future dates (enforced by application-level validation).
 7. Every login attempt (successful or failed) must be recorded in **Login_Activity** with device and location information.
 8. Accounts are automatically locked for 30 minutes after 5 consecutive failed login attempts (enforced by trigger `trg_LoginActivity_FailedAttempt`).
 9. All profile modifications must be logged in **Profile_Audit_Log** with the user ID, timestamp, and field-level before/after values in **Audit_Field_Change**.
 10. A **Diet_Plan** must maintain nutritional balance by distributing target calories and macros across its diet days and meals.
 11. Referential integrity must be enforced between all entities through foreign key constraints with appropriate ON DELETE / ON UPDATE actions.
 12. Progress reports (weekly/monthly) are generated based on data from **Progress_Log** and **Progress_Log_Exercise**.
 13. All database access from the GUI must go through stored procedures; direct SQL execution from application code is prohibited.

2.4 Entity Relationship Diagram

The following ERD represents the complete logical database design for the Virtual Fitness Trainer system using Crow's Foot notation. It illustrates all major entities, their attributes, primary keys, foreign keys, and the cardinalities of relationships between them.

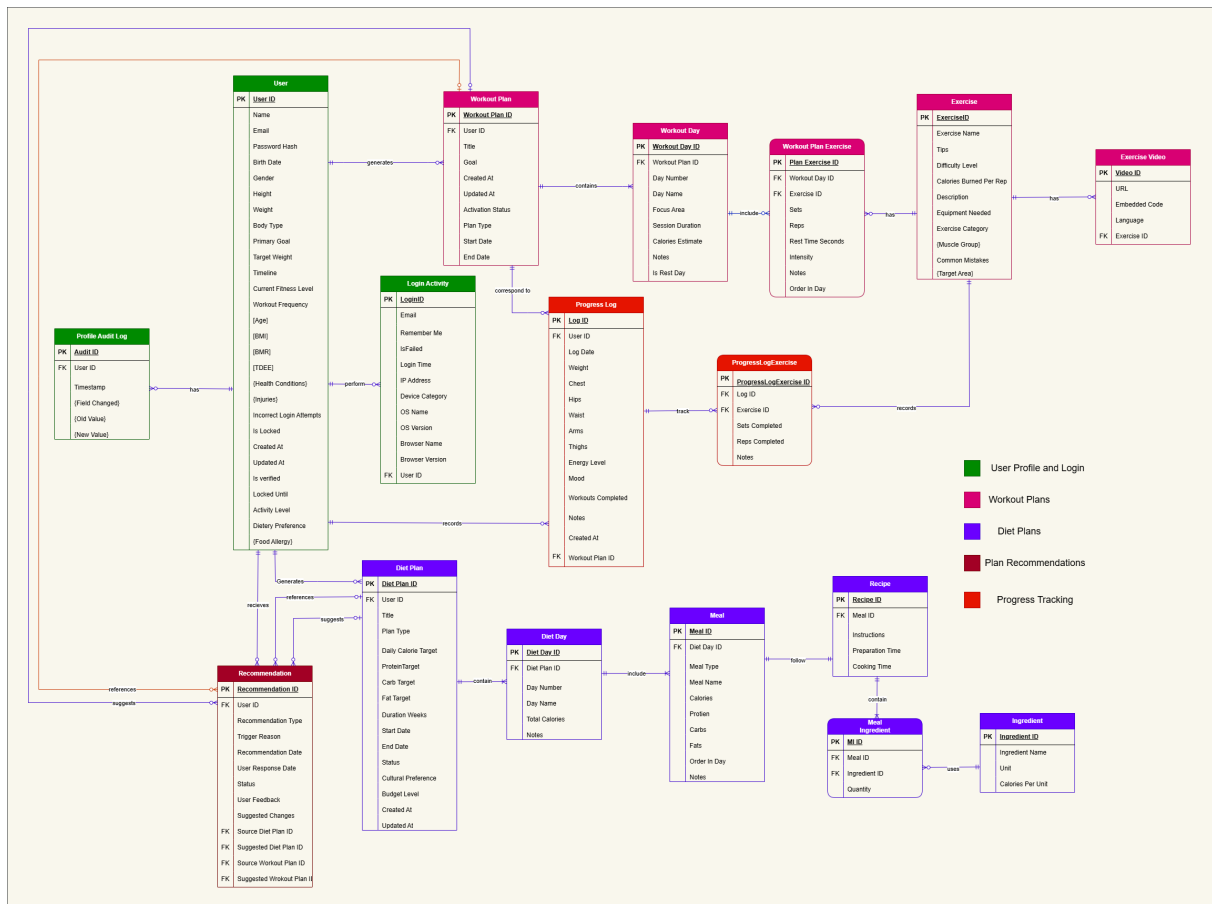


Figure 2.1. Virtual Fitness Trainer — Entity Relationship Diagram (Crow's Foot Notation)

ERD Key Relationships:

- User $\rightarrow$ Login_Activity, Profile_Audit_Log, Workout_Plan, Diet_Plan, Progress_Log, Recommendation (1:N each)
- Workout_Plan $\rightarrow$ Workout_Day (1:N) $\rightarrow$ Workout_Plan_Exercise (1:N)
- Exercise $\leftrightarrow$ Workout_Day via Workout_Plan_Exercise (M:N resolved)
- Exercise $\rightarrow$ Exercise_Video (1:N), Muscle_Group (1:N), Target_Area (1:N)
- Diet_Plan $\rightarrow$ Diet_Day (1:N) $\rightarrow$ Meal (1:N) $\rightarrow$ Recipe (1:1)
- Recipe $\leftrightarrow$ Ingredient via Meal_Ingredient (M:N resolved)
- Progress_Log $\rightarrow$ Progress_Log_Exercise (1:N)

Chapter 3

ERD to Relational Schema Conversion

3.1 Conversion Methodology

The ERD was converted to a normalized relational schema by following standard ERD-to-relational mapping rules:

1. **Strong entities** → individual relations with their attributes; the primary key of the entity becomes the relation's primary key.
2. **Weak entities** → relations whose primary key includes the foreign key of the identifying owner entity.
3. **Multivalued attributes** → separate relations with a foreign key referencing the parent entity (applied to User, Exercise, Profile_Audit_Log).
4. **Binary M:N relationships** → associative/junction relations with composite keys (Workout_Plan_Exercise, Meal_Ingredient, Progress_Log_Exercise).
5. **Binary 1:N relationships** → foreign key placed in the “many” side relation.
6. **Derived attributes** (Age, BMI, BMR, TDEE) are not stored independently; they are computed and cached in the User table for performance.

3.2 Resulting Relational Schema

This is the final relational schema:

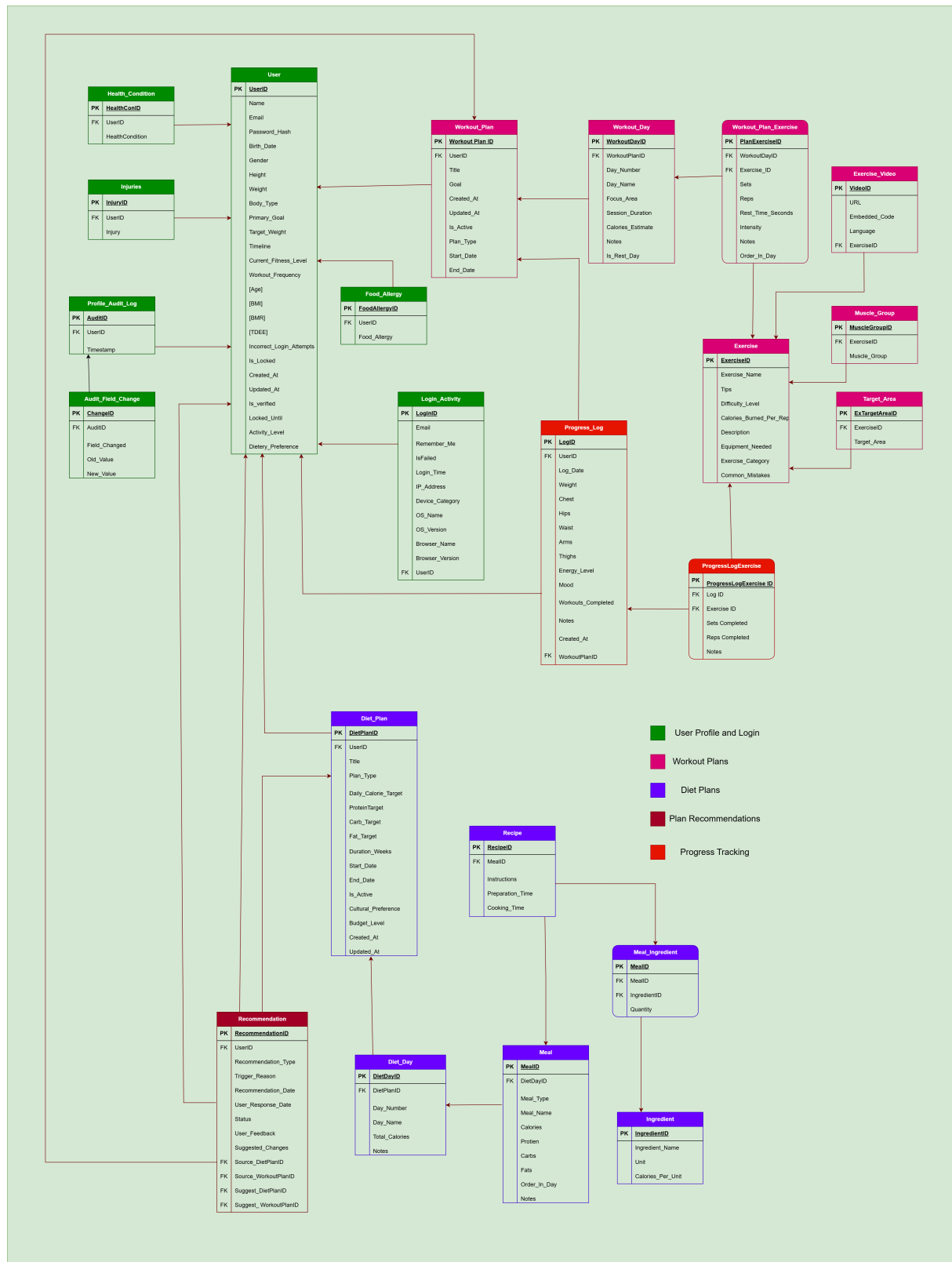


Figure 3.1. Virtual Fitness Trainer — Relational Schema

Chapter 4

Functional Dependencies and Normalization

4.1 Functional Dependencies

For each relation in the Virtual Fitness Trainer relational schema, all non-trivial functional dependencies are listed below. The notation $A \rightarrow B$ means attribute (or attribute set) A functionally determines attribute B .

4.1.1 User

$UserID \rightarrow Name, Email, Password_Hash, Birth_Date, Gender,$
 $Height, Weight, Body_Type, Primary_Goal, Target_Weight,$
 $Timeline, Current_Fitness_Level, Workout_Frequency,$
 $Incorrect_Login_Attempts, Is_Locked, Created_At,$
 $Updated_At, Is_verified, Locked_Until, Activity_Level,$
 $Dietary_Preference$
 $UserID \rightarrow [Age], [BMI], [BMR], [TDEE]$

Note: $[Age]$, $[BMI]$, $[BMR]$, and $[TDEE]$ are derived attributes.

4.1.2 Health_Condition

$HealthConID \rightarrow UserID, HealthCondition$

4.1.3 Food_Allergy

$FoodAllergyID \rightarrow UserID, Food_Allergy$

4.1.4 Profile_Audit_Log

AuditID → UserID, Changed_At, Changed_By

4.1.5 Audit_Field_Change

ChangeID → AuditID, Field_Name, Old_Value, New_Value

4.1.6 Login_Activity

LoginID → Email, Remember_Me, IsFailed, Login_Time,
IP_Address, Device_Category, OS_Name, OS_Version,
Browser_Name, Browser_Version, UserID

4.1.7 Workout_Plan

WorkoutPlanID → UserID, Title, Goal, Created_At, Updated_At,
Is_Active, Plan_Type, Start_Date, End_Date

4.1.8 Workout_Day

WorkoutDayID → WorkoutPlanID, Day_Number, Day_Name, Focus_Area,
Session_Duration, Calories_Estimate, Notes, Is_Rest_Day

4.1.9 Workout_Plan_Exercise

PlanExerciseID → WorkoutDayID, Exercise_ID, Setss, Reps,
Rest_Time_Seconds, Intensity, Notes, Order_In_Day

Composite candidate key: {WorkoutDayID, ExerciseID, Order_In_Day} → Setss, Reps, Rest_Time

4.1.10 Exercise

ExerciseID $\rightarrow$ Exercise_Name, Tips, Difficulty_Level,
Calories_Burned_Per_Rep, Description,
Equipment_Needed, Exercise_Category, Common_Mistakes

4.1.11 Exercise_Video

VideoID $\rightarrow$ URL, Embedded_Code, Language, ExerciseID

4.1.12 Muscle_Group

MuscleGroupID $\rightarrow$ ExerciseID, Muscle_Group

4.1.13 Target_Area

ExTargetAreaID $\rightarrow$ ExerciseID, Target_Area

4.1.14 Progress_Log

LogID $\rightarrow$ UserID, Log_Date, Weight, Chest, Hips,
Waist, Arms, Thighs, Energy_Level, Mood,
Notes, Overall_Progress_Notes, BMI_Snapshot

Candidate key: {UserID, Log_Date} $\rightarrow$ all non-key attributes.

4.1.15 Progress_Log_Exercise

PLogExID $\rightarrow$ LogID, ExerciseID, Sets_Completed, Reps_Completed, Notes

Composite candidate key: {LogID, ExerciseID} $\rightarrow$ Sets_Completed, Reps_Completed, Notes

4.1.16 Diet_Plan

DietPlanID $\rightarrow$ UserID, Title, Plan_Type, Goal,
Created_At, Updated_At, Is_Active, Start_Date, End_Date

4.1.17 Diet_Day

DietDayID $\rightarrow$ DietPlanID, Day_Number, Day_Name, Total_Calories_Target, Notes

Composite candidate key: {DietPlanID, Day_Number} $\rightarrow$ Day_Name, Total_Calories_Target, Notes

4.1.18 Meal

MealID $\rightarrow$ DietDayID, Meal_Name, Meal_Type, Meal_Time, Total_Calories, Notes

4.1.19 Meal_Ingredient

{RecipeID, IngredientID} $\rightarrow$ Quantity

4.1.20 Ingredient

IngredientID $\rightarrow$ Ingredient_Name, Unit, Calories_Per_Unit

4.1.21 Recipe

RecipeID $\rightarrow$ MealID, Recipe_Name, Instructions, Prep_Time, Cook_Time, Servings, Description

4.1.22 Recommendation

RecommendationID → UserID, Recommendation_Type, Trigger_Reason,
Recommendation_Date, Status, User_Response_Date,
User_Feedback, Suggested_Changes,
Source_DietPlanID, Source_WorkoutPlanID,
Suggested_DietPlanID, Suggested_WorkoutPlanID

4.2 Normalization

Each relation in the schema was examined for violations of First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF). The following subsections document the normalization steps applied to each ERD entity to yield the final set of 3NF relations.

4.2.1 User (ERD Entity: User)

Original ERD attributes. The **User** entity contained multivalued attributes {Health Conditions}, {Injuries}, and {Food Allergy}, and derived attributes [Age], [BMI], [BMR], [TDEE].

1NF. The three multivalued attributes violate 1NF. Each is decomposed into a separate relation:

- Health_Condition(HealthConID, UserID, HealthCondition)
- Injuries(InjuryID, UserID, Injury)
- Food_Allergy(FoodAllergyID, UserID, Food_Allergy)

Derived attributes [Age], [BMI], [BMR], [TDEE] are cached but computed via trigger on update.

2NF & 3NF. Single-attribute primary key UserID; no partial or transitive dependencies. In 3NF.

4.2.2 Profile_Audit_Log (ERD Entity: Profile Audit Log)

1NF. Storing multiple field-change records inside a single tuple violates 1NF. Field-level change data is extracted into **Audit_Field_Change** (one row per changed field).

2NF & 3NF. Both resulting relations have single-attribute PKs; no partial or transitive dependencies. **In 3NF.**

Resulting relations.

- Profile_Audit_Log(AuditID, UserID, Changed_At, Changed_By)
- Audit_Field_Change(ChangeID, AuditID, Field_Name, Old_Value, New_Value)

4.2.3 Login_Activity

1NF–3NF. All attributes are atomic. Single-attribute PK LoginID; no partial or transitive dependencies. **In 3NF.**

4.2.4 Exercise (ERD Entity: Exercise)

1NF. Multivalued attributes {Muscle Group} and {Target Area} violate 1NF. Decomposed into:

- Muscle_Group(MuscleGroupID, ExerciseID, Muscle_Group)
- Target_Area(ExTargetAreaID, ExerciseID, Target_Area)

2NF & 3NF. All three resulting relations have single-attribute PKs. **In 3NF.**

4.2.5 Workout_Plan, Workout_Day, Workout_Plan_Exercise

All three relations have single-attribute (or surrogate) primary keys. All non-key attributes depend solely and directly on the primary key. No partial or transitive dependencies exist. **All in 3NF.**

For **Workout_Plan_Exercise**, the composite candidate key {WorkoutDayID, ExerciseID, Order_ID} is noted; the surrogate PlanExerciseID is used as PK to eliminate any partial-dependency concern.

4.2.6 Diet_Plan, Diet_Day, Meal, Recipe

All four relations have single-attribute primary keys. All non-key attributes depend directly on their respective PKs. **All in 3NF.**

4.2.7 Meal_Ingredient

2NF. The composite PK is {RecipeID, IngredientID}. The only non-key attribute Quantity depends on the *full* composite key (the quantity of an ingredient varies per

recipe). No partial dependency. **In 2NF.**

3NF. With only one non-key attribute, transitive dependency is impossible. **In 3NF.**

4.2.8 Progress_Log and Progress_Log_Exercise

Progress_Log: PK is LogID; candidate key {UserID, Log_Date} noted. No transitive dependencies. **In 3NF.**

Progress_Log_Exercise: Composite candidate key {LogID, ExerciseID}; surrogate PK used. **In 3NF.**

4.2.9 Recommendation

Source_DietPlanID and Source_WorkoutPlanID are foreign-key references, not attributes that transitively determine other non-key attributes. **In 3NF.**

4.2.10 Normalization Summary

Relation	Final NF	Key Transformation Applied
User	3NF	Multivalued attrs removed to Health_Condition, Injuries, Food_Allergy
Health_Condition	3NF	Decomposed from User {Health Conditions}
Injuries	3NF	Decomposed from User {Injuries}
Food_Allergy	3NF	Decomposed from User {Food Allergy}
Profile_Audit_Log	3NF	Field-change detail moved to Audit_Field_Change
Audit_Field_Change	3NF	Decomposed from Profile_Audit_Log
Login_Activity	3NF	No transformation needed
Exercise	3NF	Multivalued {Muscle Group}, {Target Area} removed
Muscle_Group	3NF	Decomposed from Exercise multivalued {Muscle Group}

Relation	Final NF	Key Transformation Applied
Target_Area	3NF	Decomposed from Exercise multivalued {Target Area}
Exercise_Video	3NF	No transformation needed
Workout_Plan	3NF	No transformation needed
Workout_Day	3NF	No transformation needed
Workout_Plan_Exercise	3NF	Surrogate key added for composite candidate key
Diet_Plan	3NF	No transformation needed
Diet_Day	3NF	No transformation needed
Meal	3NF	No transformation needed
Recipe	3NF	No transformation needed
Meal_Ingredient	3NF	Composite key; Quantity depends on full key
Ingredient	3NF	No transformation needed
Progress_Log	3NF	Exercise details moved to Progress_Log_Exercise
Progress_Log_Exercise	3NF	Decomposed from Progress_Log
Recommendation	3NF	No transformation needed

Chapter 5

Database Implementation

5.0.1 Triggers

Seven triggers enforce business rules automatically:

Trigger Name	Event	Purpose
trg_ProgressLog_DateCheck	BEFORE INSERT	Prevents logging of future dates.
trg_LoginActivity_FailedAttem	AFTER INSERT	Locks account after 5 failed logins.
trg_LoginActivity_SuccessRese	AFTER INSERT	Resets failed-attempt counter on success.
trg_User_Before_Update	BEFORE UPDATE	Recalculates BMI, BMR, TDEE; logs profile changes.
trg_WorkoutPlan_DeactivateOl	BEFORE INSERT	Deactivates all existing workout plans before new one is inserted.
trg_DietPlan_DeactivateOld	BEFORE INSERT	Deactivates existing diet plans before new insertion.
trg_DietPlan_SetEndDate	BEFORE INSERT	Auto-calculates end date (Start_Date + 7 days).

```
1 CREATE TRIGGER trg_WorkoutPlan_DeactivateOld
2 BEFORE INSERT ON Workout_Plan
3 FOR EACH ROW
4 BEGIN
5     UPDATE Workout_Plan
6     SET     Is_Active = 0,
7           Updated_At = NOW()
8     WHERE  UserID    = NEW.UserID
9           AND  Is_Active = 1;
10 END$$
```

Listing 5.1. Trigger: Deactivate Old Workout Plan (excerpt)

5.0.2 Scalar Functions

Function Name	Purpose
fn_GetUserIDByEmail	Returns UserID for a given email address.
fn_IsEmailTaken	Returns 1 if email already registered, 0 otherwise.
fn_IsAccountLocked	Returns 1 if user account is currently locked.
fn_IsUserVerified	Returns 1 if user email has been verified.
fn_CalcBMI	Calculates BMI from weight (kg) and height (cm).
fn_CalcBMR	Calculates Basal Metabolic Rate using Mifflin–St Jeor equation.

5.0.3 Stored Procedures

All 38 stored procedures are organized into 12 functional sections:

Section	Topic	Key Procedures
2	User Registration & Auth	sp_RegisterUser, sp_LoginUser, sp_VerifyUserEmail, sp_AddHealthCondition, sp_AddInjury, sp_AddFoodAllergy
3	User Profile Management	sp_GetUserProfile, sp_UpdateUserProfile, sp_UpdateUserPassword, sp_DeleteUserAccount, sp_GetProfileAuditLog
4	Exercise Library	sp_GetAllExercises, sp_GetExerciseDetail, sp_AddExercise, sp_UpdateExercise, sp_DeleteExercise

Section	Topic	Key Procedures
5	Workout Plan Management	sp_CreateWorkoutPlan, sp_AddWorkoutDay, sp_AddExerciseToWorkoutDay, sp_GetWorkoutPlan, sp_GetUserWorkoutPlans, sp_DeactivateWorkoutPlan, sp_DeleteWorkoutPlan
6	Diet Plan Management	sp_CreateDietPlan, sp_AddDietDay, sp_AddMeal, sp_AddRecipeToMeal, sp_GetDietPlan, sp_GetMealIngredients, sp_GetWeeklyShoppingList, sp_DeleteDietPlan
7	Progress Tracking	sp_LogWeeklyProgress, sp_LogExerciseCompletion, sp_GetProgressLog, sp_DeleteProgressLog
8	Progress Reports	sp_GetWeeklyProgressReport, sp_GetMonthlyProgressReport
9	Recommendations	sp_CreateRecommendation, sp_GetUserRecommendations, sp_RespondToRecommendation, sp_AnalyzeProgressForRecommendation
10	Dashboard	sp_GetUserDashboard, sp_GetWorkoutStreak
11	Ingredient & Recipe	sp_AddIngredient, sp_AddIngredientToRecipe, sp_GetAllIngredients
12	Admin / Fallback Plans	sp_GetLoginActivityReport, sp_GetFallbackWorkoutPlan, sp_GetFallbackDietPlan

Chapter 6

GUI Design and Development

6.1 Technology Stack

Layer	Technology	Purpose
Backend	Python Flask 3.0	Web framework, routing, session management
Database	MySQL 8.0 + PyMySQL	Data storage and retrieval via stored procedures
AI Integration	Google Gemini AI (gemini-2.0-flash)	AI-powered workout and diet plan generation
Frontend	Bootstrap 5, Jinja2	Responsive UI, HTML templating
Security	Flask-Login, Flask-Bcrypt, Flask-WTF	Authentication, password hashing, CSRF protection
PDF Export	ReportLab	Progress report PDF generation
Video	YouTube Embedded Code	Exercise video demonstration

6.2 Application Architecture

The application follows the **Flask Blueprint** architecture, where each major feature is a self-contained module:

```
GUI Application/  
  app.py           # Application entry point  
  config.py        # Environment configuration (.env)  
  requirements.txt  # Python dependencies  
  db/
```

```

connection.py      # PyMySQL database connection factory
models.py         # Flask-Login User model
blueprints/       # Feature modules (Blueprints)
  auth/           # Login, Register, Logout, Verification
  dashboard/      # Main dashboard
  profile/        # Profile view and edit
  workout/        # AI workout plan generation and view
  diet/           # AI diet plan generation and view
  exercise/       # Exercise library with video player
  progress/       # Weekly progress logging
  reports/        # Reports and PDF export
  recommendations/ # System recommendations
templates/        # Jinja2 HTML templates
static/
  css/main.css    # Custom styles
  js/main.js      # Client-side JavaScript

```

6.3 GUI Features and Screens

6.3.1 User Registration and Login

The registration form collects all user profile information including personal details, health conditions, injuries, food allergies, and fitness goals. Passwords are hashed with bcrypt before storage. Login is handled through the `sp_LoginUser` stored procedure which validates credentials, checks account lock status, and records the login attempt in `Login_Activity`.

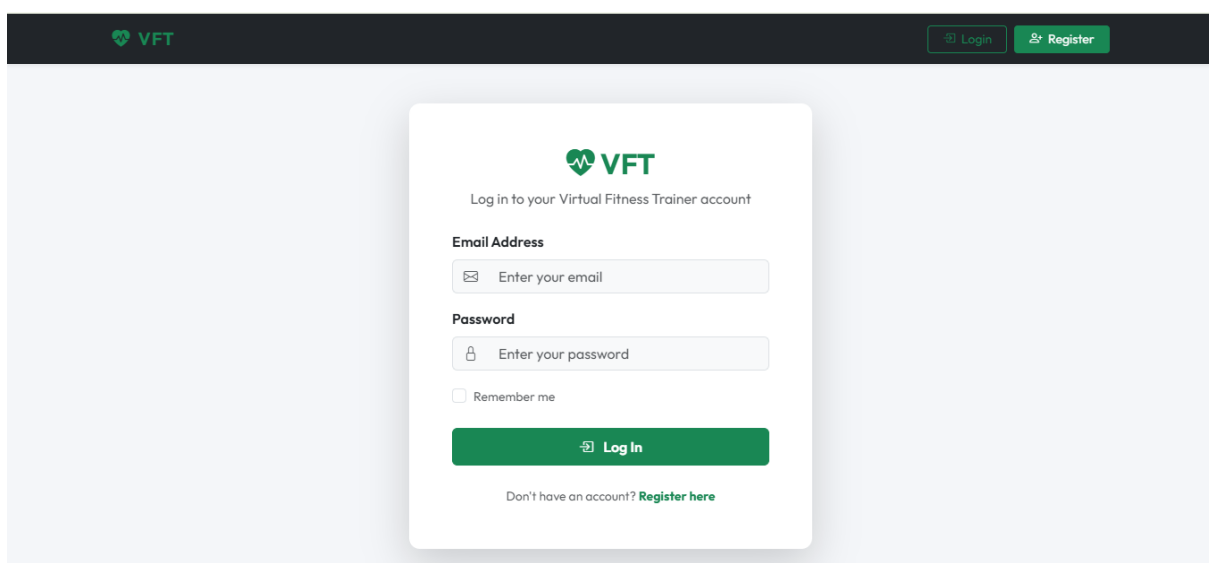
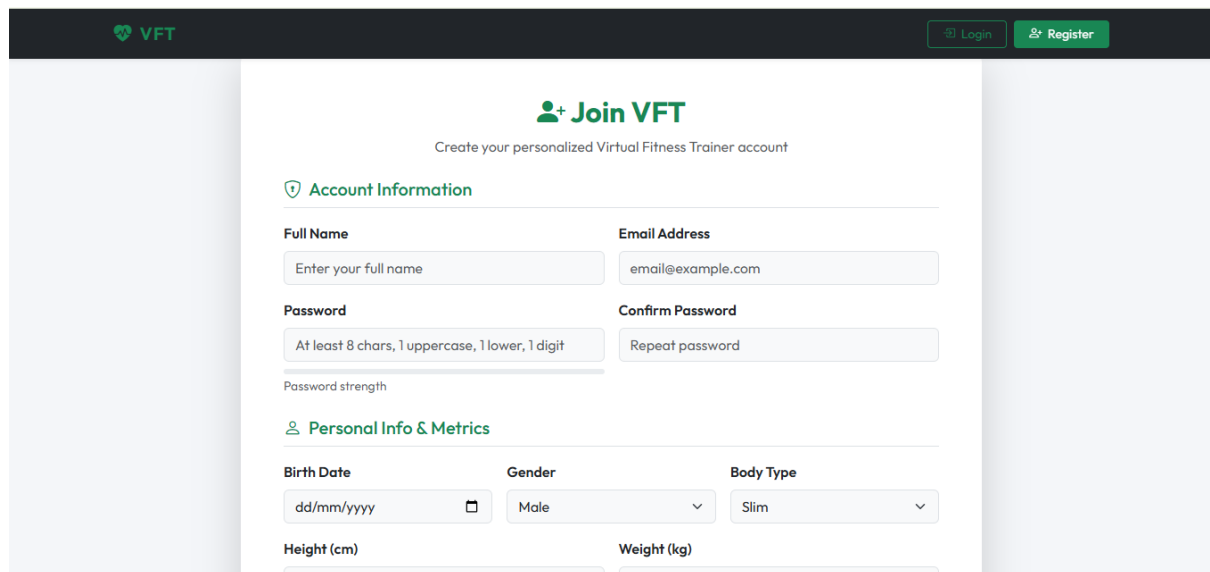


Figure 6.1. Virtual Fitness Trainer — Login Page



The registration page features a dark header with the VFT logo and 'Login'/'Register' buttons. The main content area is titled 'Join VFT' with a sub-header 'Create your personalized Virtual Fitness Trainer account'. It is divided into two sections: 'Account Information' and 'Personal Info & Metrics'. The 'Account Information' section contains fields for 'Full Name', 'Email Address', 'Password', and 'Confirm Password', with a password strength indicator. The 'Personal Info & Metrics' section includes fields for 'Birth Date', 'Gender', 'Body Type', 'Height (cm)', and 'Weight (kg)'.

Figure 6.2. Virtual Fitness Trainer — Registration Page

6.3.2 Dashboard

The central dashboard displays:

- Active workout and diet plan summaries
- Key health metrics: BMI, BMR, TDEE
- Most recent progress log entry
- Pending AI recommendations
- Recent login history

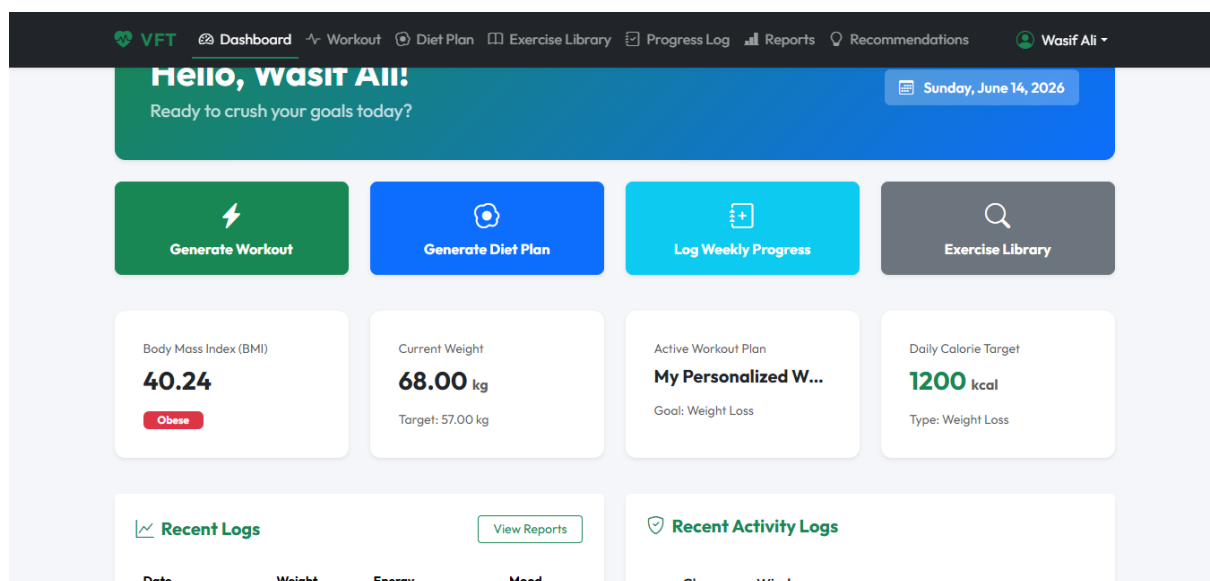


Figure 6.3. Virtual Fitness Trainer — Dashboard

6.3.3 AI Workout Plan Generation

Users can generate a personalized 7-day workout plan through the Google Gemini AI API. The application sends the user's health profile, goals, fitness level, and preferences to the AI model and stores the returned plan via `sp_CreateWorkoutPlan`, `sp_AddWorkoutDay`, and `sp_AddExerciseToWorkoutDay`. If the AI API is unavailable, the system automatically falls back to pre-built template plans using `sp_GetFallbackWorkoutPlan`.

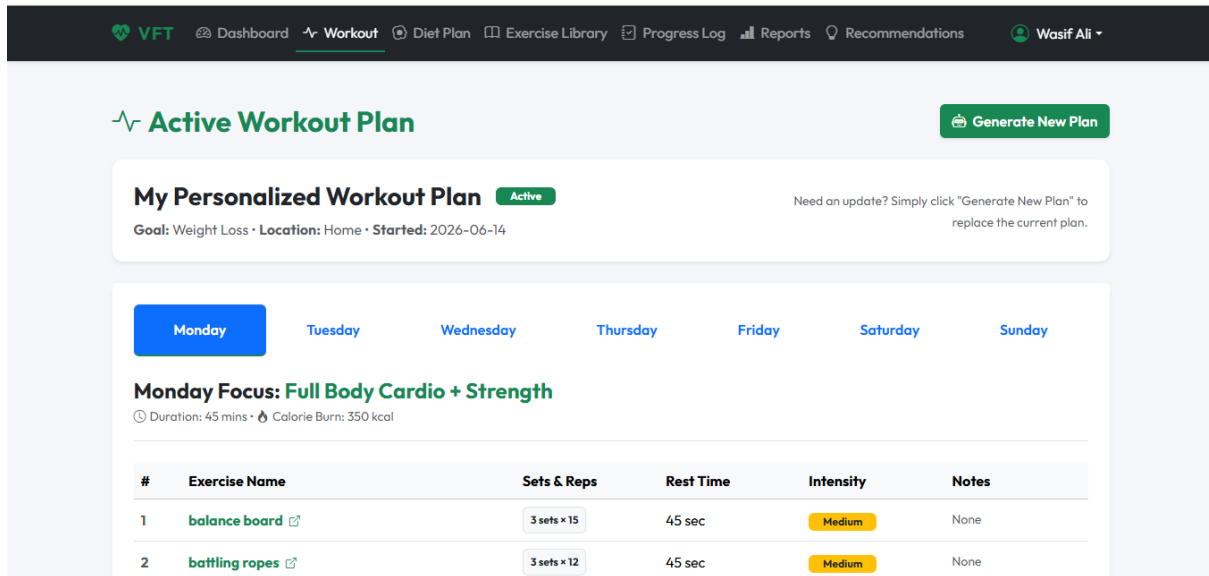


Figure 6.4. Virtual Fitness Trainer — Workout Plan Page

6.3.4 AI Diet Plan Generation

Similarly, diet plans are generated by the Gemini AI and stored via `sp_CreateDietPlan`, `sp_AddDietDay`, `sp_AddMeal`, and `sp_AddRecipeToMeal`. Users can view a complete weekly shopping list generated by `sp_GetWeeklyShoppingList`. Clicking any meal shows the full recipe and ingredient breakdown.

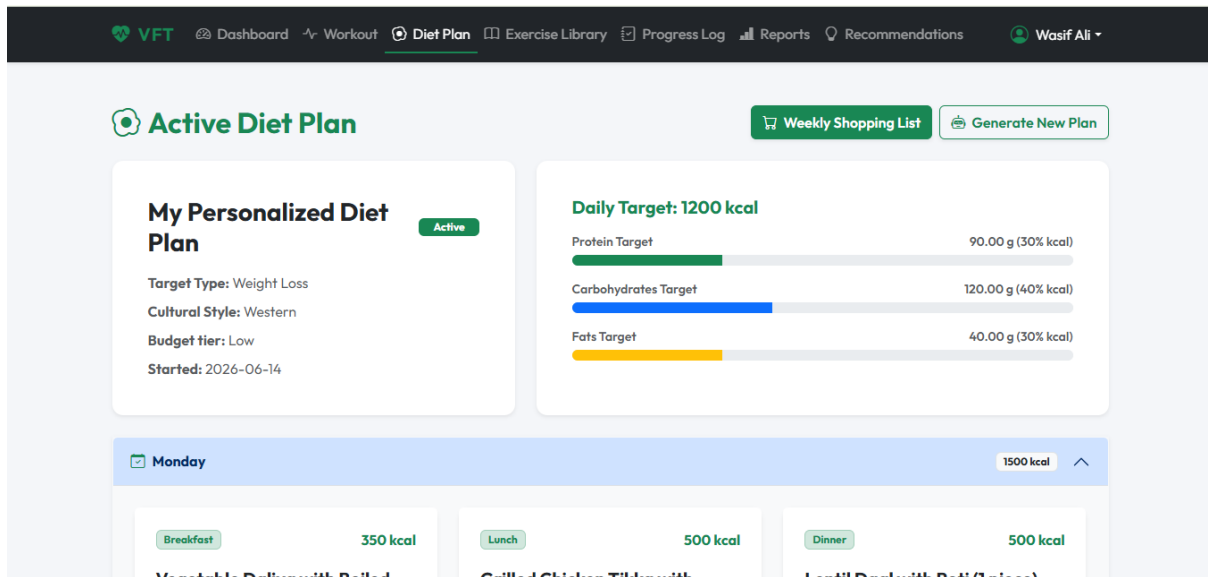


Figure 6.5. Virtual Fitness Trainer — Diet Plan Page

6.3.5 Exercise Library

The exercise library provides a searchable and filterable catalog of all exercises. Filters include category (Cardio, Strength, Flexibility, HIIT, Yoga, Pilates), difficulty level, and equipment required. Clicking any exercise card opens a detail panel showing description, tips, common mistakes, target muscle groups, and an **embedded YouTube video player** using the `Embedded_Code` stored in the `Exercise_Video` table. An external YouTube link is also provided.

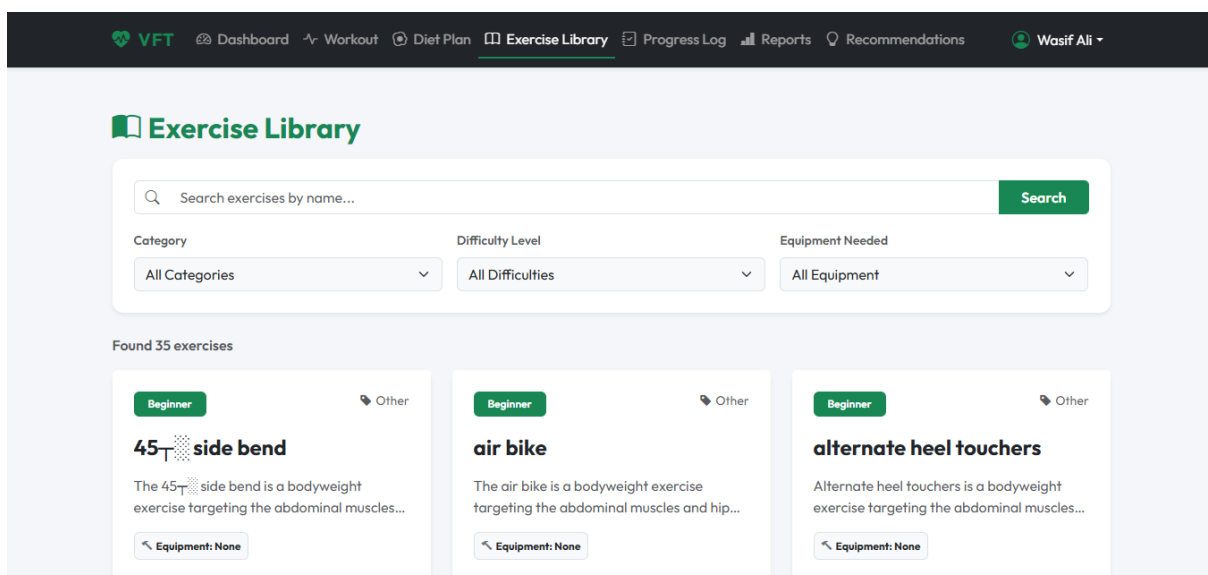


Figure 6.6. Virtual Fitness Trainer — Exercise Library

6.3.6 Weekly Progress Logging

Users can log their progress **once per week**. The progress form collects:

- Current weight and body measurements (chest, waist, hips, arms, thighs)
- Self-reported energy level (1–5) and mood
- Number of workouts completed
- Optional notes

The weekly exercise completion log allows users to check off which exercises they completed (stored in **Progress_Log_Exercise** via `sp_LogExerciseCompletion`).

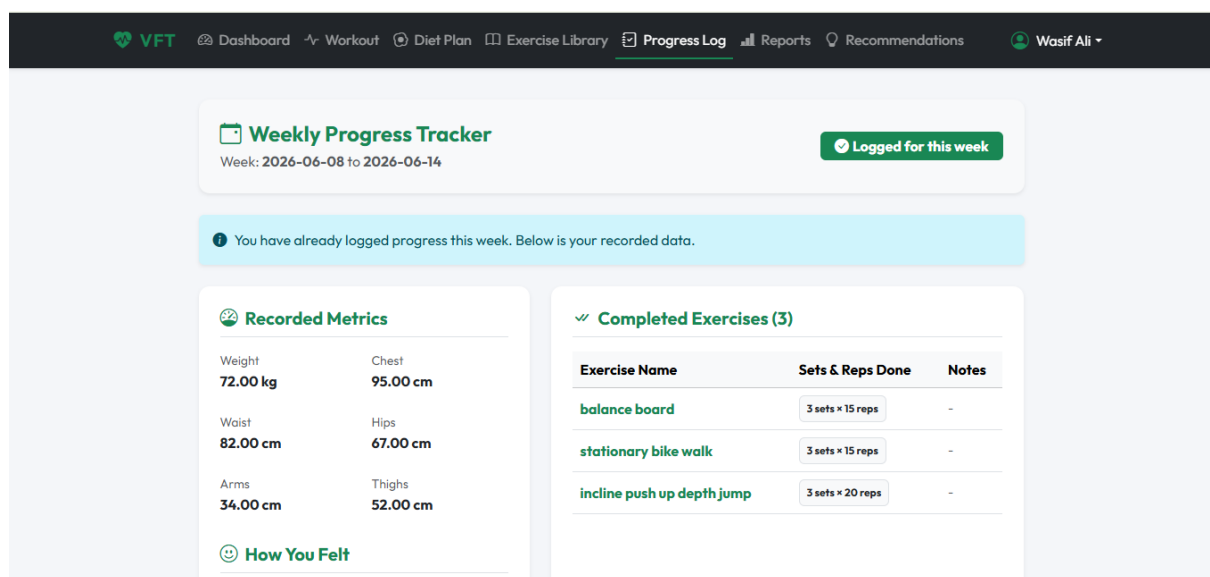


Figure 6.7. Virtual Fitness Trainer — Progress Log Page

6.3.7 Progress Reports and PDF Export

Users can view weekly or monthly progress summaries by selecting a date range. Reports show average weight, measurement trends, energy levels, and workout completion rates. Reports can be exported as formatted PDF files using ReportLab.

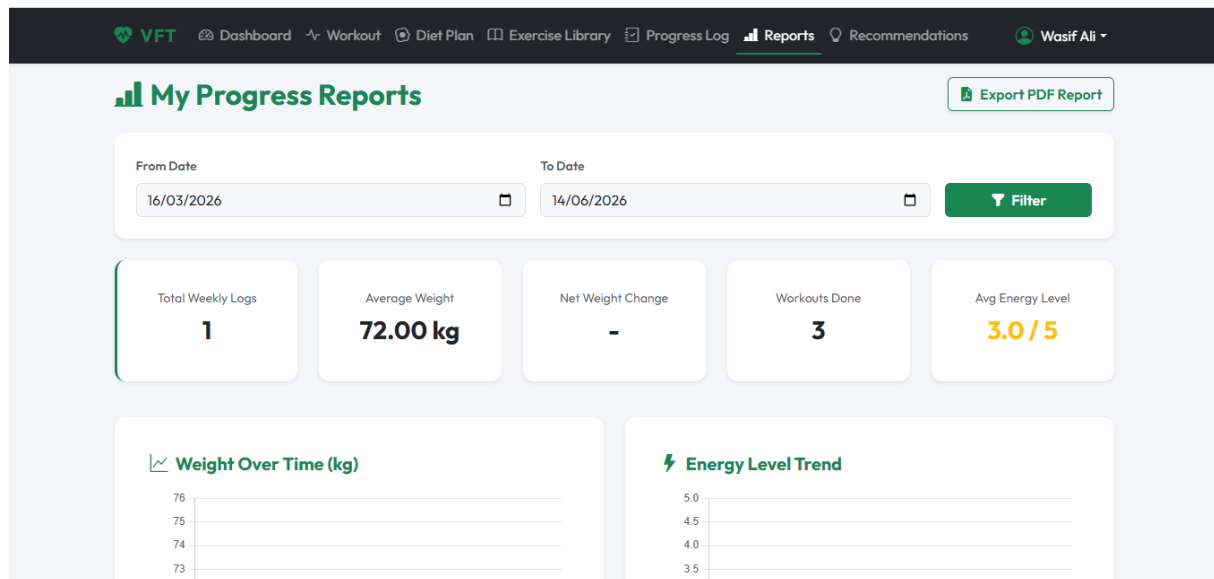


Figure 6.8. Virtual Fitness Trainer — Progress Reports and PDF Export Page

6.3.8 Recommendations

The recommendation engine generates AI-driven suggestions when progress deviates significantly from plan targets. Users can accept, modify, or dismiss recommendations through `sp_RespondToRecommendation`.

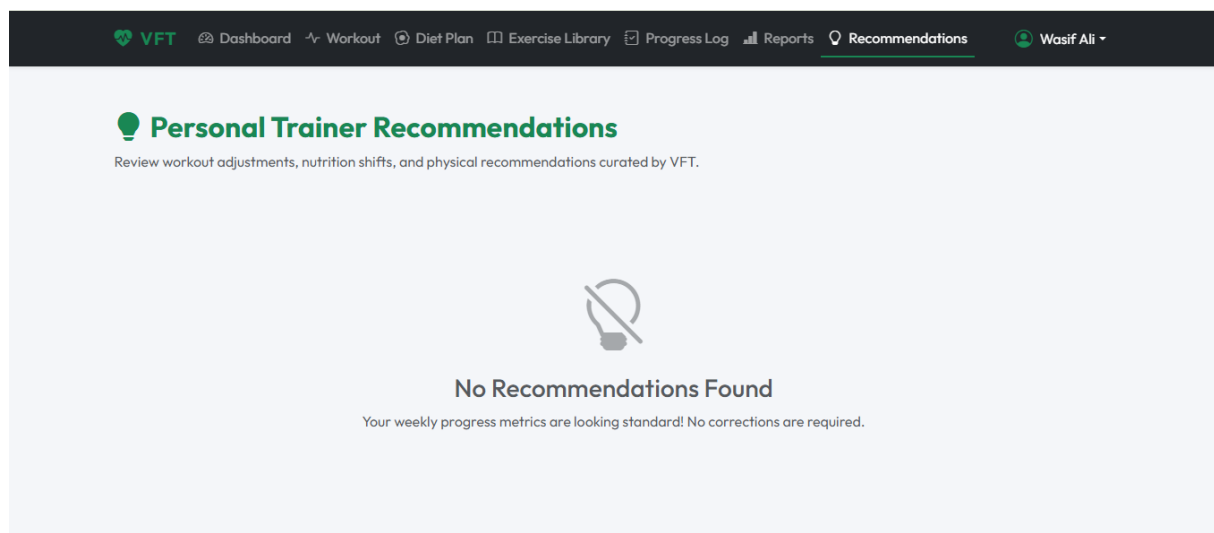


Figure 6.9. Virtual Fitness Trainer — Recommendations Page

6.4 Database Connectivity

Key Design Principle: All database operations in the Flask application are performed exclusively through stored procedures. No raw SQL queries are executed directly from the application code, ensuring secure, consistent, and structured data access.

A typical stored procedure call from the Flask backend:

```
1 # blueprints/workout/routes.py
2 def get_user_workout_plan(user_id):
3     with get_db_connection() as conn:
4         with conn.cursor() as cursor:
5             cursor.callproc('sp_GetWorkoutPlan', [workout_plan_id])
6             plan_rows = cursor.fetchall()
7     return plan_rows
```

Listing 6.1. Flask: Calling Stored Procedure (example)

6.5 Security Implementation

- **Password Security:** All passwords hashed with bcrypt (Flask-Bcrypt) before storage.
- **CSRF Protection:** All forms protected with CSRF tokens via Flask-WTF.
- **Account Lockout:** Accounts locked for 30 minutes after 5 consecutive failed login attempts (trigger-enforced).
- **Session Management:** Flask-Login manages secure user sessions.
- **Secrets:** Database credentials and API keys stored in `.env` file, excluded from version control.
- **Audit Logging:** All profile changes are logged in `Profile_Audit_Log` and `Audit_Field_Change` via trigger.

Chapter 7

References

1. M. D. Mifflin, S. T. St Jeor, L. A. Hill, B. J. Scott, S. A. Daugherty, and Y. O. Koh, “A new predictive equation for resting energy expenditure in healthy individuals,” *American Journal of Clinical Nutrition*, vol. 51, no. 2, pp. 241–247, Feb. 1990.
2. Muhammad Wasif, Muhammad Shahid, and Hifza Bibi, “Software Requirements Specification for Virtual Fitness Trainer,” Version 2.0, Department of Computer Science, Namal University, Mianwali, Pakistan, Dec. 2025.
3. Project Proposal: Virtual Fitness Trainer, Department of Computer Science, Namal University, Mianwali, Nov. 2025.
4. BodBot, “BodBot Personal Trainer,” Google Play Store, 2025. [Online]. Available: <https://play.google.com/store/apps/details?id=com.bodbot>
5. Leap Fitness Group, “Home Workout – No Equipment,” Google Play Store, 2025.
6. Anthropic, “Claude,” Large Language Model, 2025. [Online]. Available: <https://www.anthropic.com>
7. xAI, “Grok,” Large Language Model, 2025. [Online]. Available: <https://x.ai>
8. Google, “Gemini AI (gemini-2.0-flash),” Large Language Model, 2025. [Online]. Available: <https://aistudio.google.com>
9. Kaggle, “Exercise and Workout Datasets,” *Kaggle Datasets*, 2024–2025. [Online]. Available: <https://www.kaggle.com> [Used for workout plans and diet plan sample data in *dbDML.sql*.]
10. YouTube (Google LLC), “Exercise Video Demonstrations,” 2025. [Online]. Available: <https://www.youtube.com> [YouTube video URLs and embedded iframe codes used in the *Exercise_Video* table for in-app video demonstrations.]
11. Kaggle, “Nutrition and Diet Plan Datasets,” *Kaggle Datasets*, 2024–2025. [Online]. Available: <https://www.kaggle.com> [Used for ingredient, meal, and recipe sample data in *dbDML.sql*.]

Appendix A

Appendix: AI Usage and Prompts

AI Tools Used

- **Grok** (by xAI) — <https://x.ai>
- **Claude** (by Anthropic) — <https://www.anthropic.com>
- **Google Gemini AI** — <https://aistudio.google.com> (integrated in the application itself)

Purpose of AI Usage

AI tools were used to assist in database implementation, stored procedure development, GUI application generation, error debugging, normalization documentation in LaTeX, DML data generation, and README writing. All AI-generated content was reviewed, validated, and adapted by the project team.

Prompts Used During Milestone 3

The following table documents all prompts submitted to AI tools during Milestone 3 development.

Sr.	Prompt (Paraphrased)	Purpose
1	Act as best prompt engineer for generating complete application... write a detailed prompt for AI to use Flask (excluding SQLAlchemy) to create the GUI... requirement change: user logs progress on weekly basis.	Initial GUI generation prompt for AI IDE.
2	While running app: <code>ModuleNotFoundError: No module named 'flask'</code> — provide step-by-step solution.	Flask module installation fix.
3	I have provided you the GUI application zip... the app is running on localhost but after login it causes errors... provide the corrections.	Post-login error debugging.

Sr.	Prompt (Paraphrased)	Purpose
4	After making your suggested changes, registration works but login shows an error about invalid parameters for a procedure... provide a detailed prompt for antigravity describing all procedure details.	Procedure-call correction in Flask code.
5	Act as expert in normalization of ERD and LaTeX... provide LaTeX code for Functional Dependencies and Normalization sections for listed tables.	Normalization documentation generation.
6	I have provided the final dbDDL.sql... make dbDML.sql with minimum 30 rows per table using the provided CSV datasets for exercises, diet plans, and workout plans.	DML data generation.
7	ERROR 1442 (HY000): Can't update table 'workout_plan' in stored function/trigger — why does this error occur and what is the solution?	Trigger conflict debugging.
8	Design and develop GUI... All database operations must be performed through stored procedures. Provide procedures.sql with all necessary procedures and functions.	Stored procedures creation.
9	Our instructor suggested a change: user must log progress once per week. Update progress log procedures/functions accordingly.	Weekly progress requirement update.
10	Act as expert in prompt engineering... gemini API is not working, exercise library is not showing videos, reports causing errors — provide a detailed prompt for AI to fix these issues.	Multi-error debugging prompt.
11	I have updated all procedures... on some procedures it causes errors: ERROR 1308: LEAVE with no matching label — remove the errors.	Stored procedure syntax fix.
12	I have the valid exercise video data (URL, embedded code, language) — provide updated INSERT query for exercise_video table.	Exercise video data insertion.
13	Errors: view profile shows 'Injury' error; diet plan has no recipe view; exercise uses link instead of embedded video. Write a prompt for antigravity to fix these.	GUI feature corrections.

Sr.	Prompt (Paraphrased)	Purpose
14	I have provided the complete source code of VFT — provide me the README.md following the specified format so any person can easily set up the app.	README file generation.
15	Act as best LaTeX documentator... read the provided files... make the report for Milestone 3 strictly following the database design document template.	Final Milestone 3 report generation.

Virtual Fitness Trainer — Database Design Document (Version 2.0)

CSC-271 Database Systems, Namal University, Mianwali

Muhammad Wasif (NUM-BSCS-2024-59) Hifza Bibi (NUM-BSCS-2024-27)